CSSE3113 – Introduction to Software Engineering

Software Architectural Design

Software Architecture

“The architecture of a system is a comprehensive framework that describes its form and structure – its components and how they fit together.”

Jerrold Grochow

Architectural Design

- Architectural design is concerned with understanding how a system should be organized and designing the overall structure of that system.

- It is the first stage in the software design process.

- Architectural design is a creative process where you design a system organization that will satisfy the functional and non-functional requirements of a system.

Architectural Design

These requirements include performance, security, availability, and maintainability. …

Performance: Performance can be enhanced by localizing operations to minimise sub-system communication. That is, try to have self-contained modules as much as possible so that inter-module communication is minimized.

Security: Security can be improved by using a layered architecture with critical assets put in inner layers.

Architectural Design

Availability: building redundancy in the system and having redundant components in the architecture can ensure Availability.

Maintainability: Maintainability is directly related with simplicity. Therefore, using fine-grain, self-contained components can increase maintainability.

Architectural Design Process

This involves performing a number of activities, not necessarily in any particular order or sequence.

These include

- System structuring
- Control modeling
- Modular decomposition

Architectural Design Process

System structuring: System structuring is concerned with decomposing the system into interacting sub-systems.

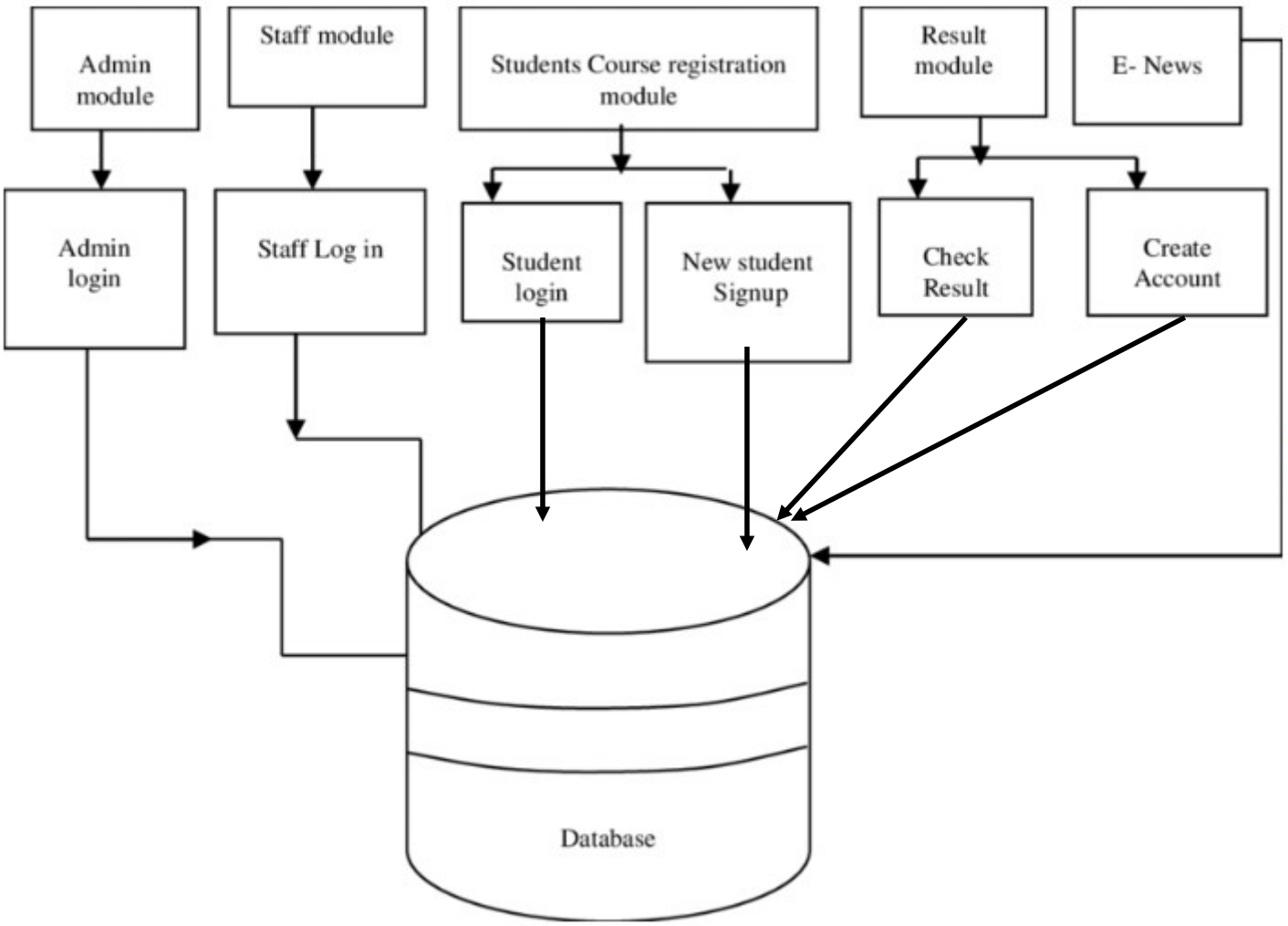
Control modeling*:* Control modelling establishes a model of the control relationships between the different parts of the system.

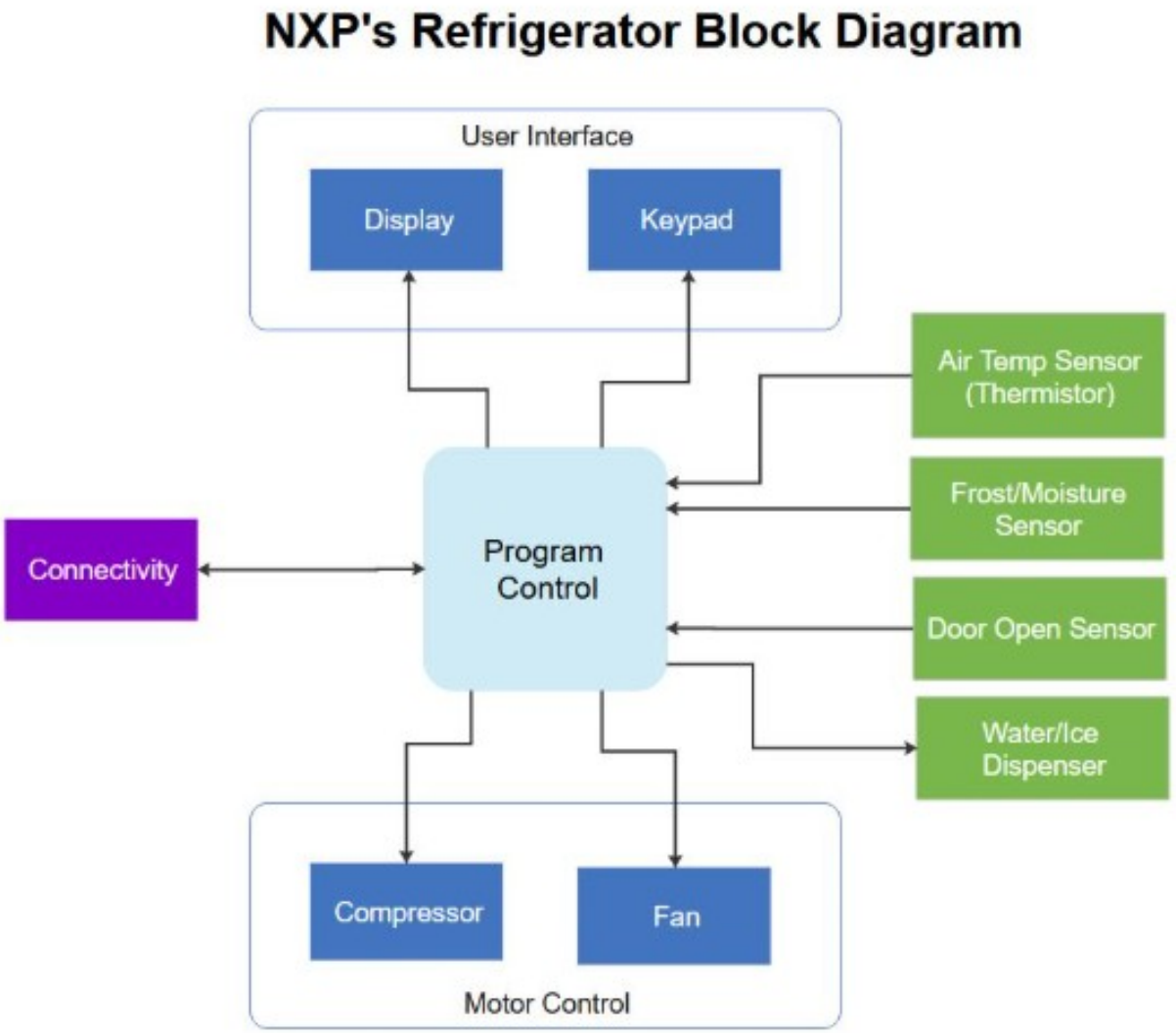
Modular decomposition*:* During this activity, the identified sub-systems are decomposed into modules.

Block Diagrams

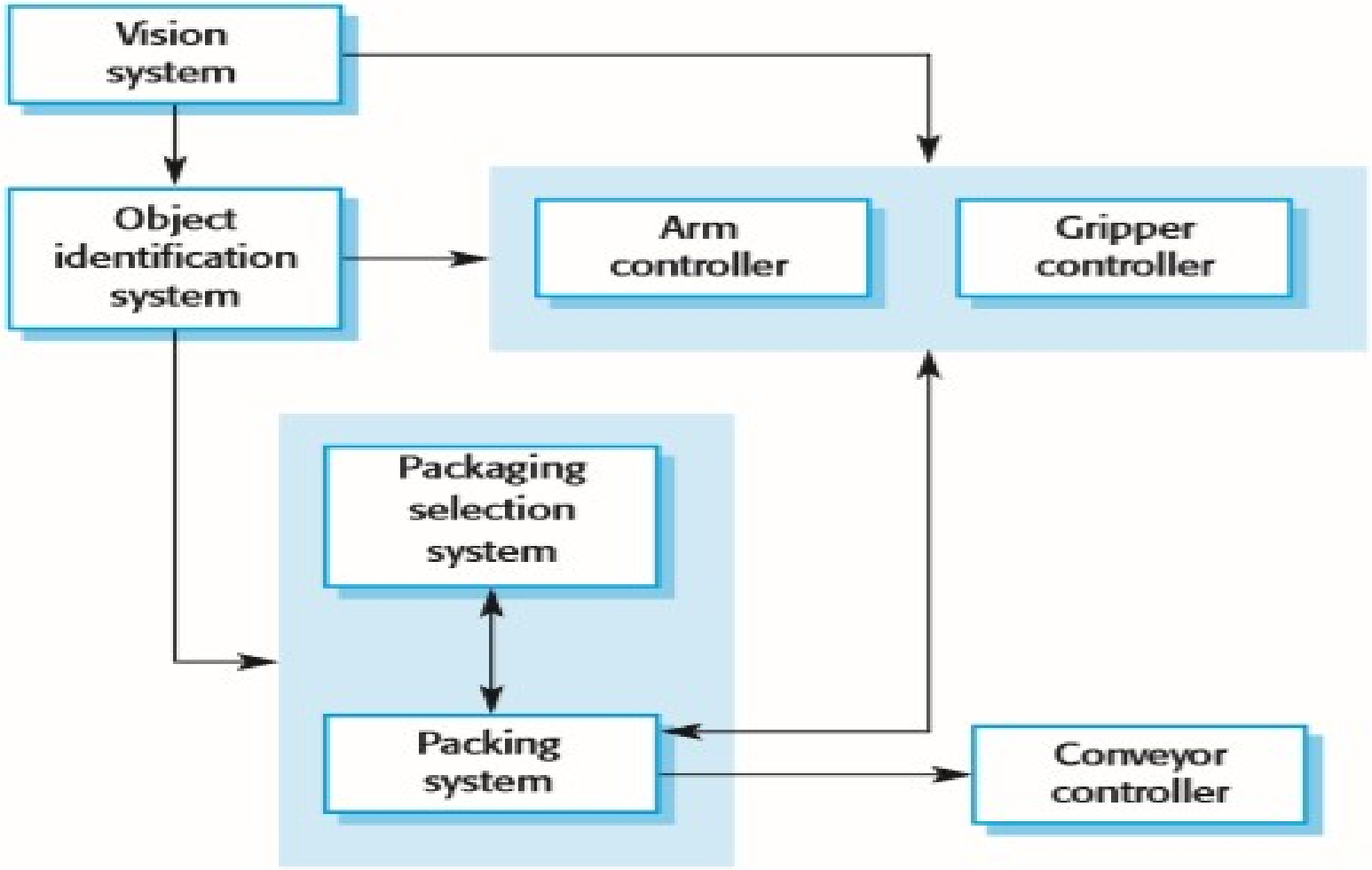
Block diagrams are often used to describe sub-system designs

- **Each box** in the diagram represents a **sub-system**.
- **Boxes within boxes** indicate that the **sub-system has itself been decomposed to sub-systems.**
- **Arrows** mean that **data and or control signals** are passed from sub-system to sub-system in the direction of the arrows.
- **Block diagrams** present **a high-level picture** of the system structure,


Admin module
Staff module
Students Course registration module
Result module
E- News
Admin login
Staff Log in
Student login
New student Signup
Check Result
Create Account
Database


NXP's Refrigerator Block Diagram
User Interface
Display
Keypad
Air Temp Sensor (Thermistor)
Frost/Moisture Sensor
Door Open Sensor
Water/Ice Dispenser
Connectivity
Program Control
Compressor
Fan
Motor Control

Block diagram of a packing robot control system



Architectural Styles

Among many styles, the most commonly practiced ones are the following:

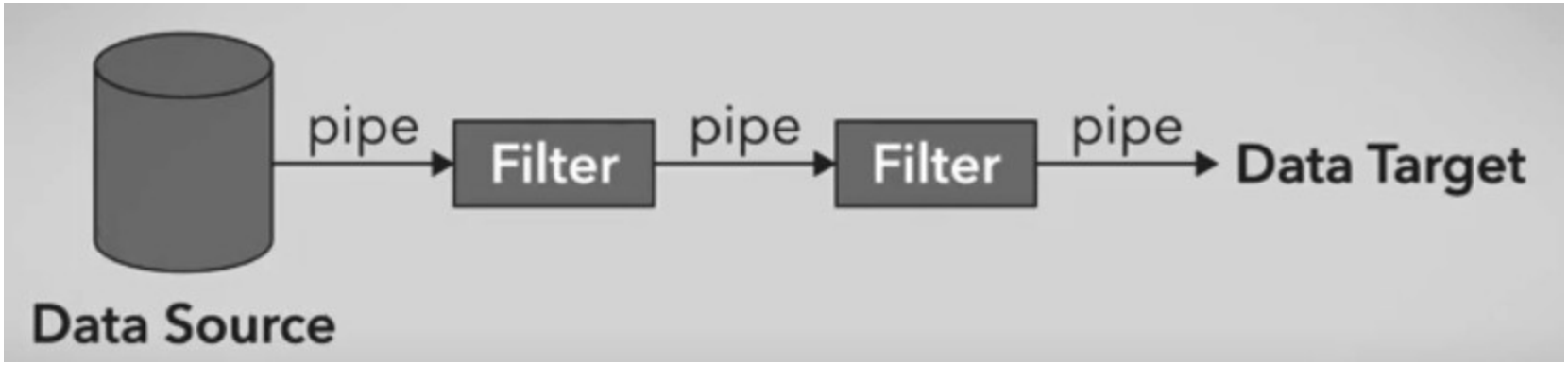
1. Pipe & Filters Architecture
2. Client Server Architecture
3. Layered Architecture
4. Model View Controller (MVC)
5. Data-Centered and Repository Architecture

Pipe and Filters Architecture

- It is a data flow architecture
- Data flow consider the system a series of transformation on set of data
- Data is transformed from one form to another using different types of operations
- The pipe-filter architecture has an independent entity call filer (which is responsible to perform operations)

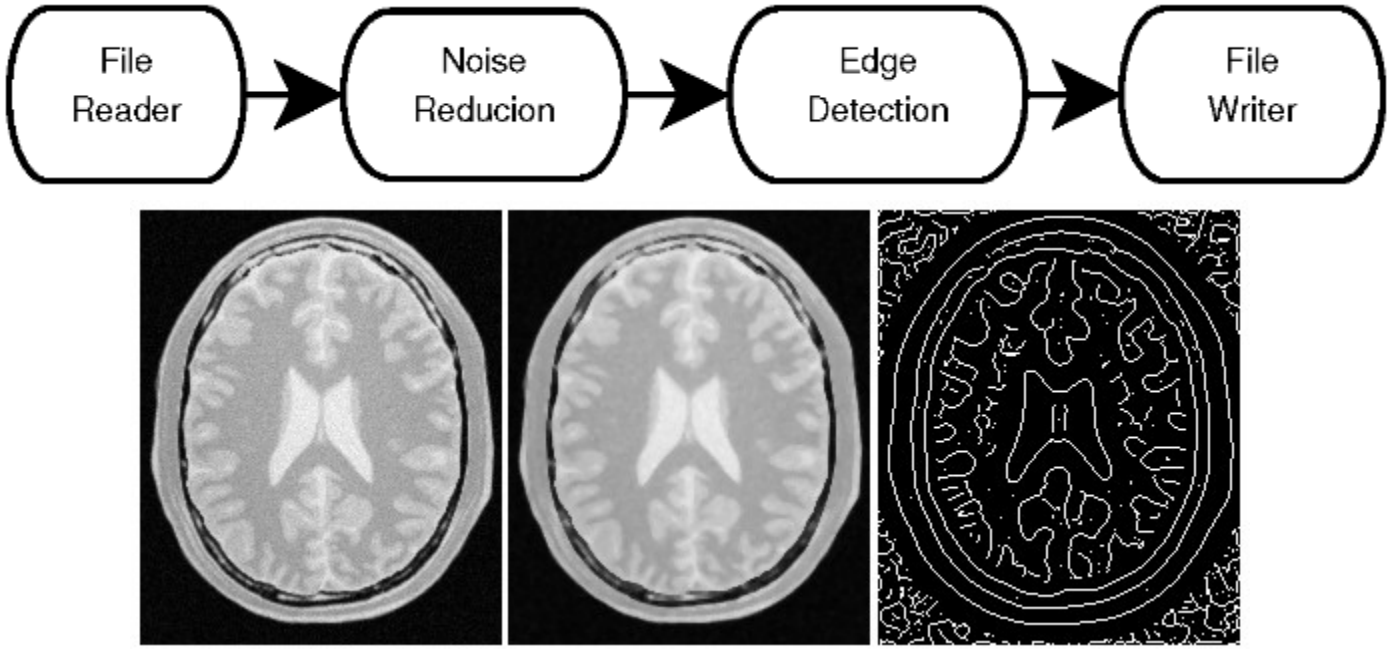
Pipe and Filter Architecture

- Filter perform operations on data input they received
- Pipes are service connectors through which data is being transformed
- Data is changed filter to filter
- In this architecture, order of the filter might effect the results.



Pipe and Filters Architecture - Example

- The output of one filter becomes the input to the others
- Example



Pipe and Filters Architecture

- Advantages:
  - Loose coupling allows filters to be changed without modifications to other filters
  - Can be develop with parallel processing.
- Disadvantages:
  - **Not a good choice for an interactive system**
  - Addition of a large number of independent filters may reduce performance due to excessive computational overheads

Examples

- **Compiler Design** –
Source code passes through filters such as *lexical analysis* → *syntax analysis* → *semantic analysis* → *code generation*.
- **Audio/Video Processing Systems** –
Raw media data passes through filters like *decoder* → *equalizer* → *compressor* → *output renderer* for transformation and playback.

Client/Server Model

- The client-server model is a distributed system model that shows how data and processing is distributed across a range of components.
- The functionality of the system is organized into services, with each service delivered from a separate server.
- Clients are users of these services and access servers to make use of them.

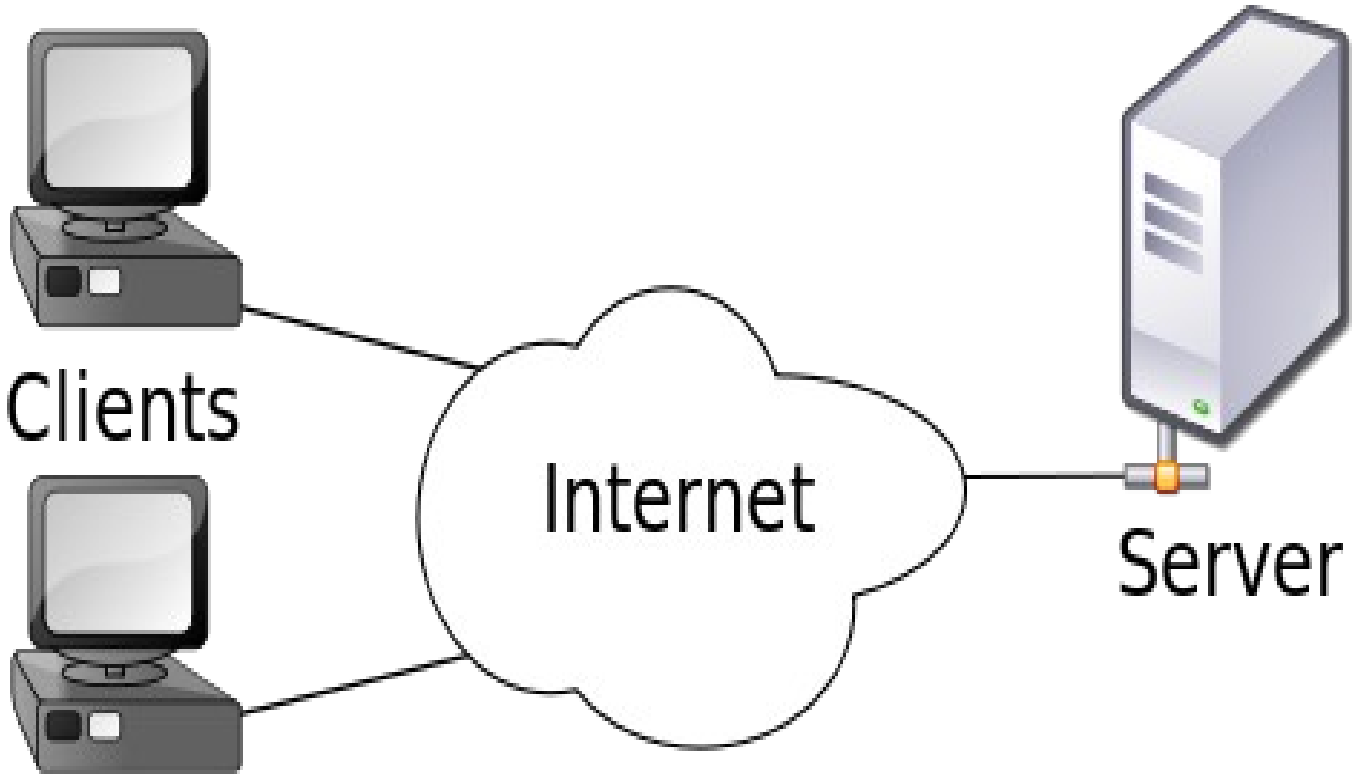
In this model, the application is modeled as

1) A set of services that are provided by servers
2) A set of clients that use these services.

Client/Server Model

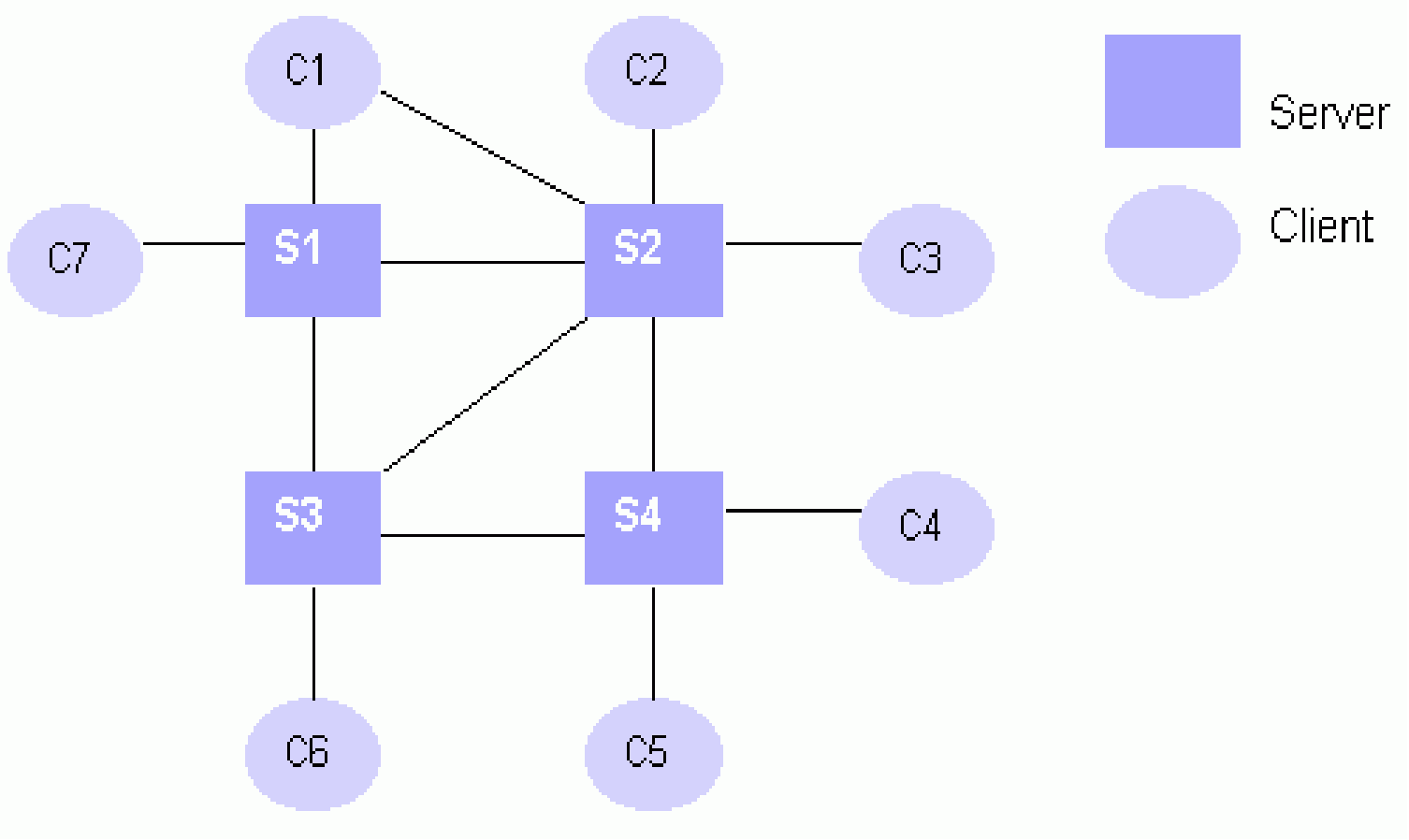
- The system is organized as a set of servers, which provide specific services such as printing, data management, etc. and a set of clients, which call on these services.
- These clients and servers are connected through a network which allows clients to access servers.
- Clients know the servers but the servers do not need to know all the clients.

Client/Server Model



Client/Server Model

General client-server organization



Client/Server Model

Following are some of the representative server types in client-server systems:

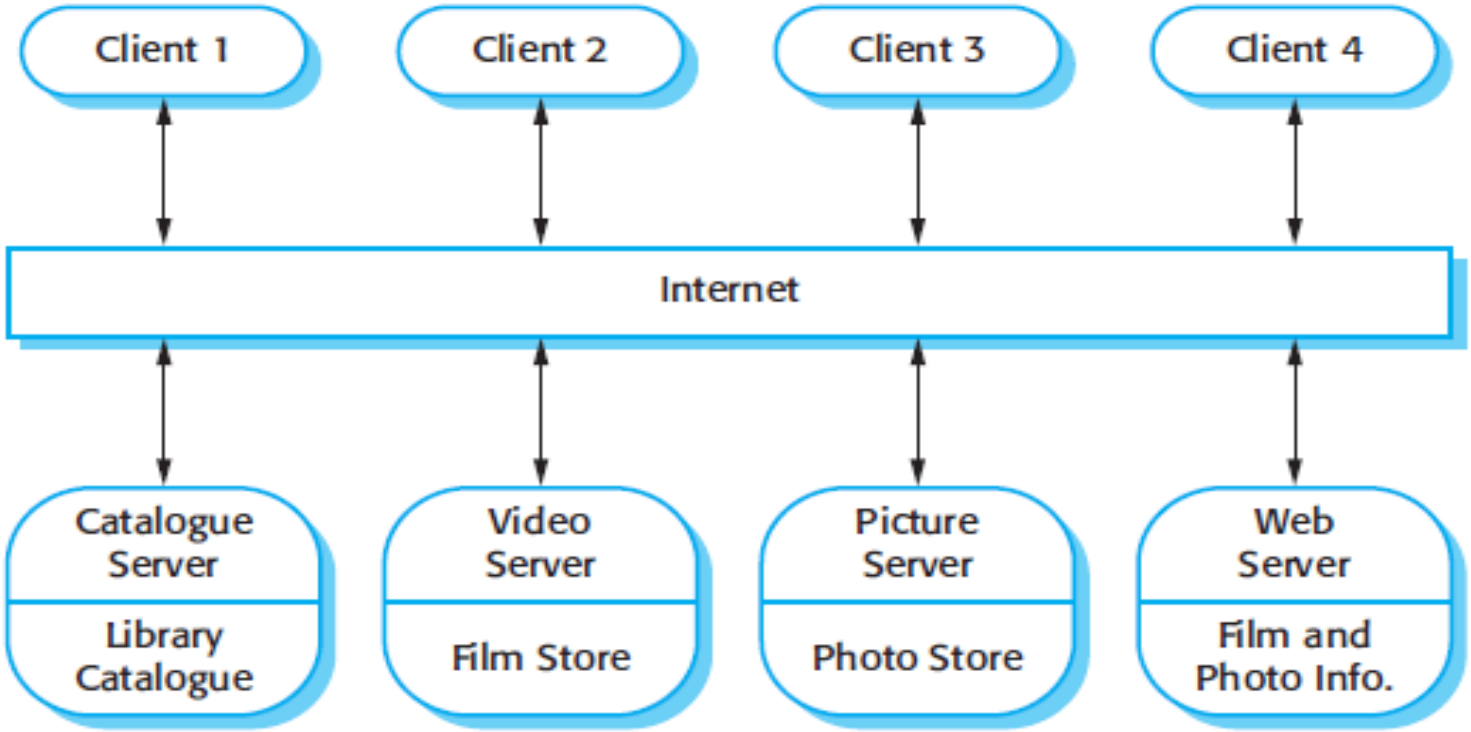
Database servers: In this case, the client sends requests, such as SQL queries, to the database server; the server processes the request and returns the results to the client over the network.

Client/Server Model

Transaction servers: In this configuration, client sends requests that invoke remote procedures on the server side; server executes procedures invoked and returns the results to the client.

Groupware servers: Groupware servers provide set of applications that enable communication among clients using text, images and video, etc.

A Client–Server Architecture for a Film Library



Client/Server Model

Advantages:

- Adding new servers or upgrading existing servers becomes easier.
- Distribution of data is straightforward in this case.

Disadvantages:

- Each service is a single point of failure so susceptible to denial of service attacks or server failure.

- Performance may be unpredictable because it depends on the network.

Layered Architecture

- A **layered architecture** organizes a system into a **set of layers** each of which provide a set of services to the layer “above”and serving as client to the layer “below”.
- Inner layers are closer to the machine hardware than the outer layers and each layer isolates the outer layer from inner complexities
- Supports incremental development

Layered Architecture

- The outer layer only needs to know the interface provided by the inner layer
- If there are any changes in the inner layer, the outer layer is not affected as long as the interface does not change
- The layered architecture pattern is another way of achieving separation and independence.

Interaction Between Layers

- Interactions among layers are defined by suitable communication protocols.
- Normally layers are ***constrained*** so elements only see:
  - other elements in the same layer, or
  - elements of the layer below

Interaction Between Layers

Flow/Protocol

- **requests** from higher layer to lower layer
- **answers** from lower layer to higher layer (**callbacks)**
- incoming data or event notification from low to high

Advantages

Independence

- Different components of the application can be **independently** deployed, maintained, and updated, on different time schedules
- Makes possible for **team members to work in parallel** on different parts of the application with minimal **dependencies.**
- Testing the components **independently** of each other.

Advantages

More secure

- Each layer may hide private information from other layers

Reusability

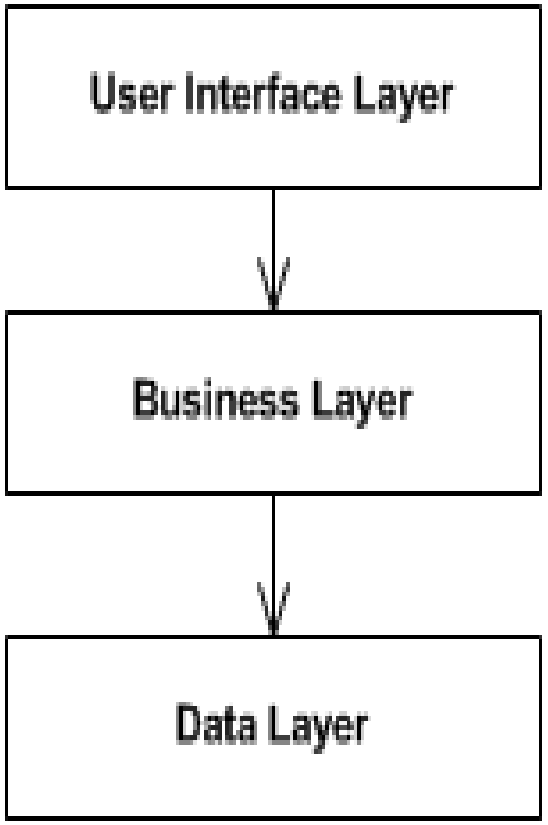
- Each layer, being cohesive and is coupled only to lower layers, makes it easier for reuse by others and easier to be replaced or interchanged

Disadvantages

- Performance degrades if we have too many layers (extra overhead of passing through layers and also changes will pass slowly to higher layers)
- Sometimes difficult to cleanly assign functionality to the “right” layer

Layered Architecture

- User interface layer responsible for displaying information and handling interaction.
- The business layer who is responsible with performing actual system functions.
- Data layer who is responsible for storing and retrieving data.



Layered Architecture

Presentation tier

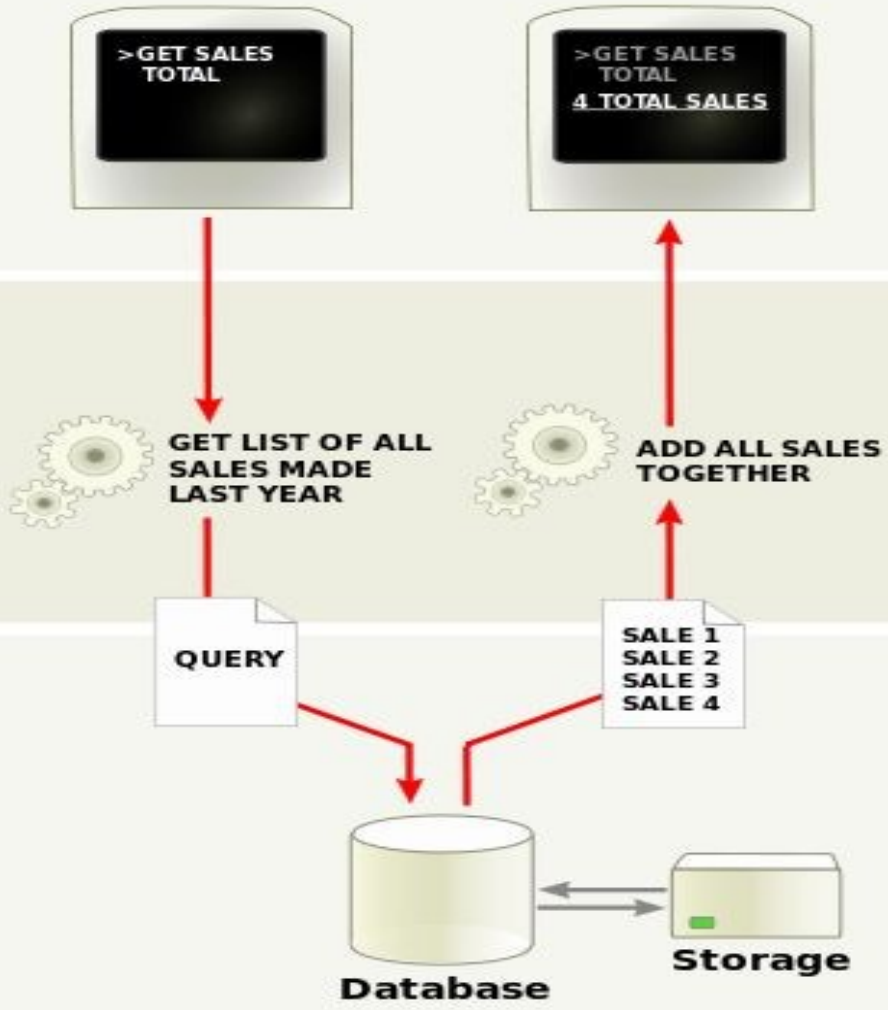
The top-most level of the application is the user interface. The main function of the interface is to translate tasks and results to something the user can understand.

Logic tier

This layer coordinates the application, processes commands, makes logical decisions and evaluations, and performs calculations. It also moves and processes data between the two surrounding layers.

Data tier

Here information is stored and retrieved from a database or file system. The information is then passed back to the logic tier for processing, and then eventually back to the user.

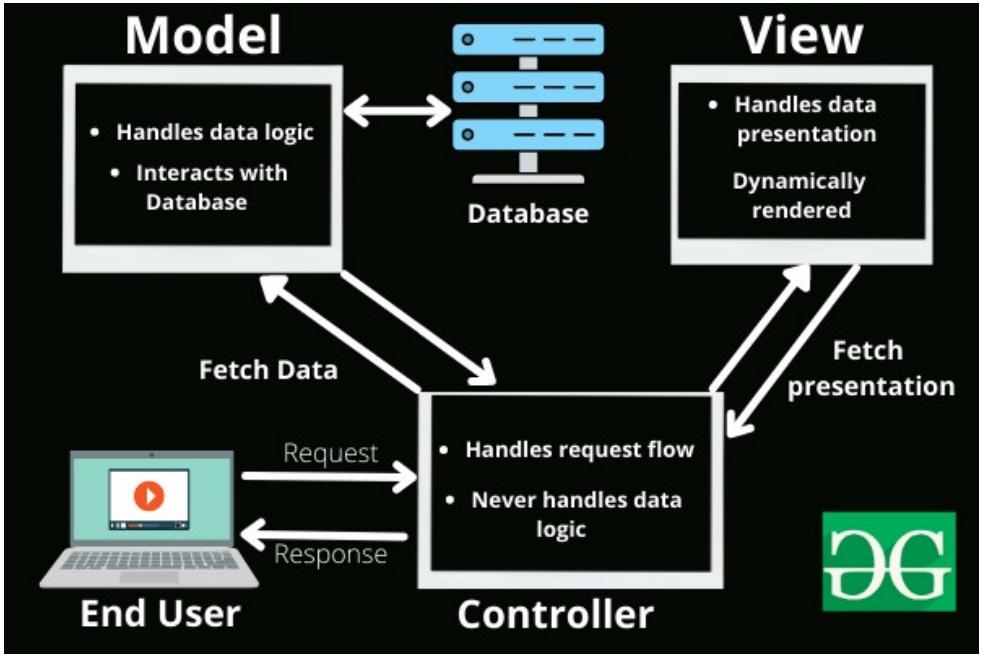


Model View Controller

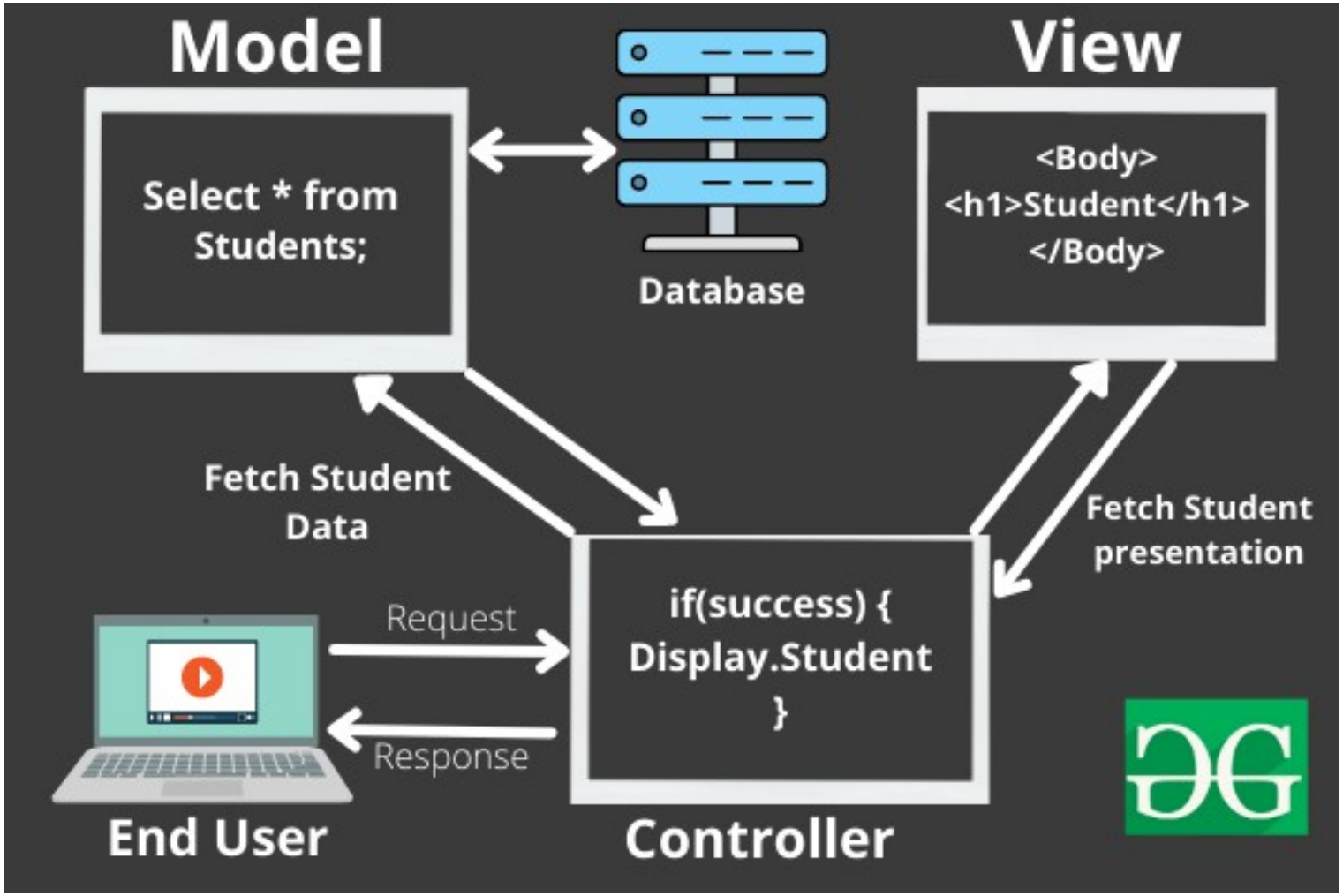
- The purpose is to decompose the application into multiple components, where every component has its own purpose
- It divide the application into three main components:
  - Model
  - View
  - Controller

Model View Controller

- This is done to separate internal representations of information from the ways information is presented to and accepted from the user.
- Popular examples of MVC frameworks:
  - **Ruby on Rails**.
  - **Django**
  - **CakePHP**



Example:


Model
Select * from Students;
Database
View
<Body>
<h1>Student</h1>
</Body>
Fetch Student Data
Fetch Student presentation
if(success) {
Display.Student
}
Request
Response
End User
Controller

Advantages and Disadvantages

Advantages:

- Separation of concern
- Easy to maintain
- MVC model returns data without formatting, so it is useful for multiple types of devices (mobile and desktop)

Disadvantages:

- **It can’t be suitable for small applications** which has adverse effect in the application's performance and design

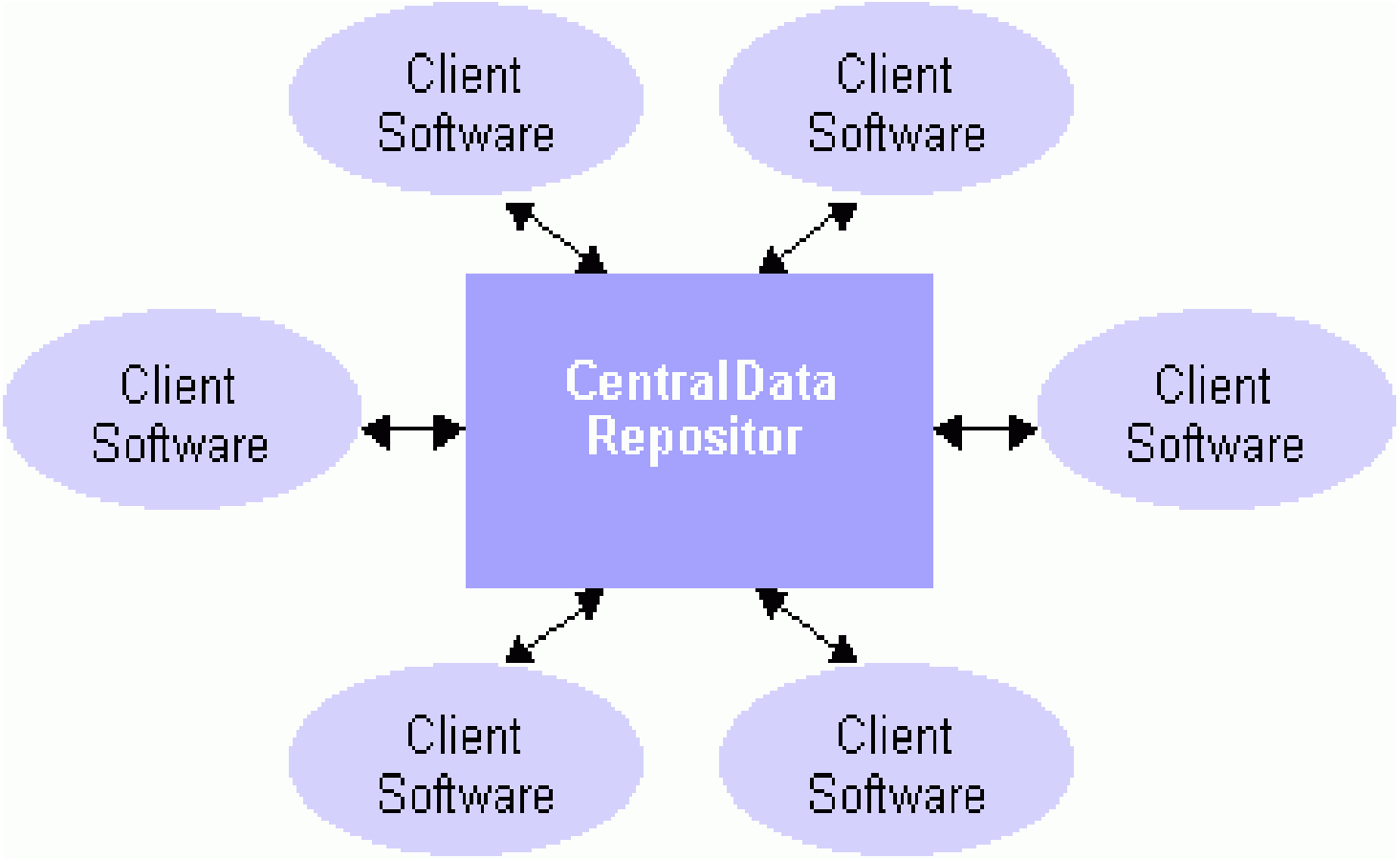
Data-Centered or Repository Architecture

- In any system, sub-systems need to exchange information and data. This may be done in two ways:

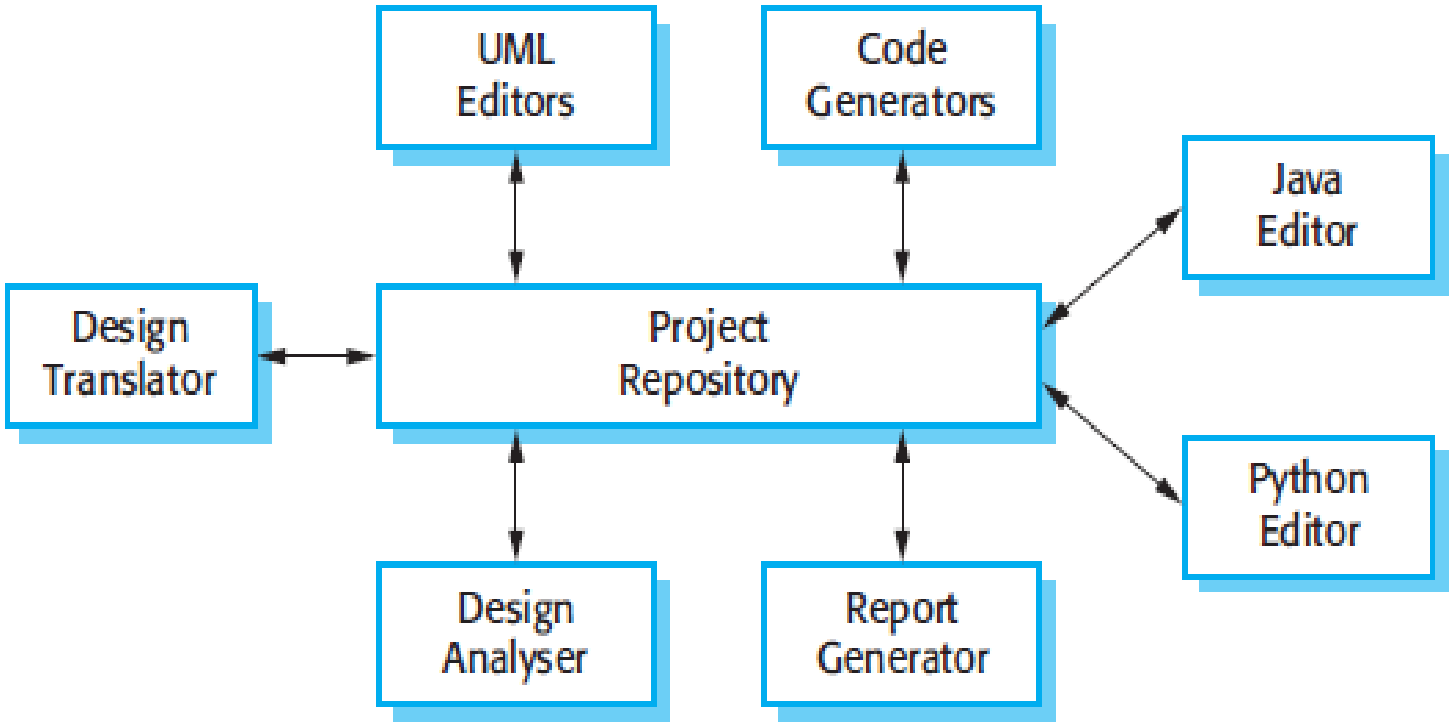
  1. Shared data is held in a **central database or repository** and may be accessed by all sub-systems
  2. Each sub-system maintains its **own database** and passes data to other sub-systems

- When large amounts of data are to be shared, the repository model of sharing is most commonly used.

Data-Centered or Repository Architecture



A Repository Architecture for an IDE



Data-Centered or Repository Architecture

- **Advantages**
  - An efficient way to share large amounts of data.
  - The sub-systems need not be concerned with how data is produced and the centralized management e.g. backup, security, etc.

- **Disadvantages**
  - The repository is a single point of failure so problems in the repository affect the whole system.